



TRACKBUS AI

Technical guide and project defense notes

VISION	CLOUD	STATUS
YOLO + ByteTrack	Workers + D1	Pilot prototype

Current evidence boundary: the end-to-end pipeline works, but the latest informal video observation suggests only about 20-30% passenger recall. This is not a validated production accuracy result.

Prepared for Favi / IMMAC
Version 1.0 | 21 August 2026

Project: TrackBus transport intelligence platform Prepared for: Favi / IMMAC Guide version: 1.0 Project version: Vision 0.2.1 / hosted gateway 0.5.0 Date: 21 August 2026

Purpose: help the project owner install, run, test, demonstrate, explain, and honestly defend the current TrackBus prototype in front of technical reviewers.

Thirty-second explanation

TrackBus is a vendor-neutral transport intelligence platform. It accepts anonymous passenger-count events from approved sources such as APC sensors, optional edge cameras, payments, imports, or manual corrections. The current Vision prototype uses YOLO to detect people, ByteTrack to maintain temporary track IDs, and a three-zone state machine to confirm boarding or alighting. Confirmed events are sent as authenticated JSON to a Cloudflare Worker, stored idempotently in D1, and shown in the Live Pilot dashboard. Raw video stays on the edge by default and the system does not perform facial recognition.

What is real and what is demonstration data

Area	Current status
Video and webcam person detection	Real YOLO inference
Temporary tracking	Real ByteTrack tracking
Boarding and alighting logic	Real OUTSIDE -> DOOR -> INSIDE state machine
JSON ingestion API	Real authenticated endpoint
Database	Real Cloudflare D1 persistence
Live Pilot event list	Real stored events, refreshed every two seconds
Forecast and dispatch scenarios	Synthetic showcase examples
Production counting accuracy	Not validated

Honest current limitation

The latest informal video test appeared to recognize only about 20-30% of visible passengers and missed many people. This should be described as an informal observation of low recall, not as a formal accuracy result. No labeled evaluation dataset was used for that estimate. The current build is acceptable as an architecture and connectivity prototype, but it is not ready for production counting claims.

1. Product idea and scope

The problem

Public transport operators often have separate systems for passenger sensors, payment validation, fleet movement, and planning. Those systems may report incompatible fields and do not automatically produce one trustworthy occupancy picture. TrackBus provides a common data contract and an operational layer above those sources.

The proposed outcome

- Estimate current passenger occupancy per bus.
- Identify crowded buses and weak data sources.
- Keep a recent, auditable event history.
- Reconcile sensor counts with other sources instead of silently forcing them to agree.
- Later forecast demand and recommend operator actions.
- Eventually expose crowding bands to a passenger application.

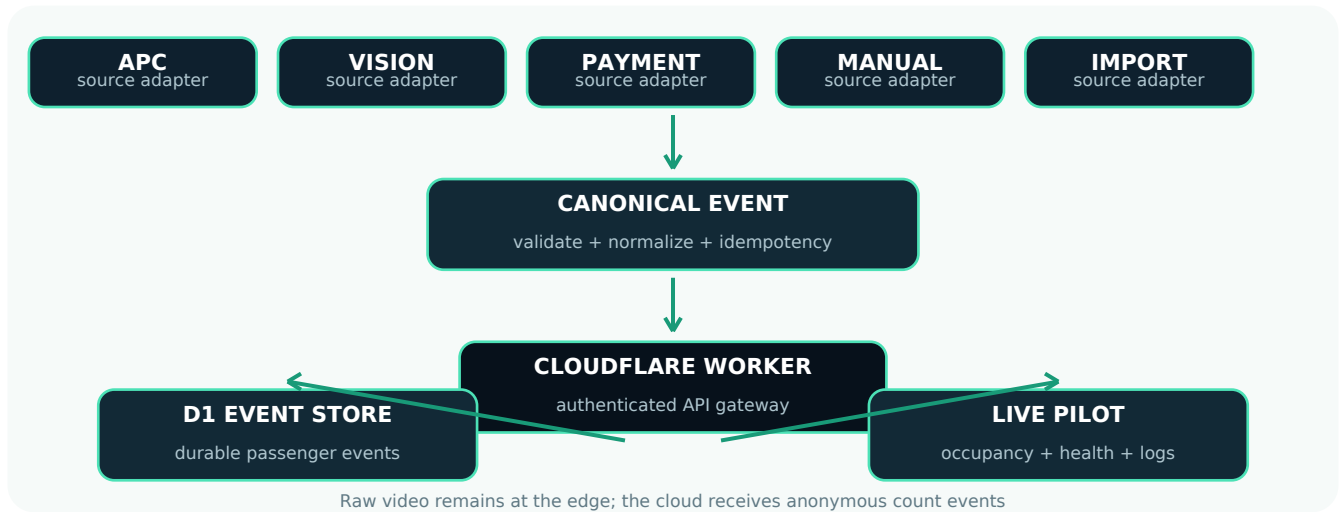
Product boundaries

- TrackBus is APC-first and vendor-neutral; Vision is optional.
- It counts anonymous movements, not identities.
- It does not use facial recognition.
- It does not send raw camera frames to the website.
- It does not directly control safety-critical vehicle systems.
- Forecast and dispatch views remain labeled synthetic until representative data validates them.

Repository map

Path	Responsibility
<code>showcase.py</code>	Main video or camera demonstration program
<code>trackbus/</code>	Detection, tracking, zones, counting, diagnostics, API client
<code>configs/</code>	Model, tracker, camera, zone, and experiment settings
<code>contracts/</code>	Canonical passenger event JSON Schema
<code>app/</code>	Web dashboard and Worker-compatible API routes
<code>components/</code>	Dashboard interface components
<code>lib/</code>	Shared TypeScript contracts, persistence helpers, sample data
<code>drizzle/</code>	Cloudflare D1 database migration SQL
<code>services/analytics-api/</code>	Independent analytics API reference service
<code>tools/</code>	Replay and pilot utilities
<code>tests/</code>	Vision, API, HTML, tracking, and trajectory verification
<code>docs/</code>	Architecture, deployment, privacy, evaluation, and pilot notes

2. End-to-end architecture



Data path in plain language

1. A local video file or camera frame is read by OpenCV.
2. Ultralytics YOLO detects objects classified as people.
3. ByteTrack associates detections across frames and gives each person a temporary track ID.
4. A configured anchor point for each track is mapped into OUTSIDE, DOOR, or INSIDE polygons.
5. The state machine confirms only a stable complete crossing.
6. TrackBus updates occupancy and creates one canonical passenger-count event.
7. The API client sends the JSON event with a bearer secret.
8. The Cloudflare Worker validates the secret and event fields.
9. D1 stores the event using `eventId` as an idempotency key.
10. The dashboard polls operational endpoints and shows new events, source health, and bus occupancy.

Why this separation matters

Detection, tracking, counting, transport events, storage, and presentation are separate boundaries. A future physical APC sensor can replace the camera without rewriting the dashboard. A different database can replace D1 without rewriting the Vision state machine. This is the main meaning of vendor-neutral in the current design.

Degraded connectivity

If the API is unavailable, the Vision client writes canonical event files under `data/offline-queue/`. It retries the same event ID later. Because the server treats `eventId` as an idempotency key, a retry does not create a second passenger count.

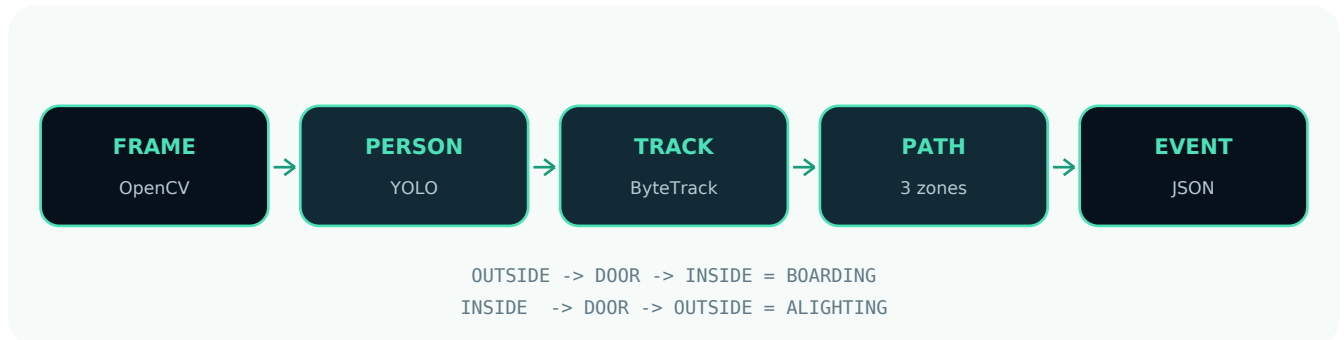
3. Technology stack and why it was used

Technology	Role	Why it fits the prototype
Python 3.11+	Vision and analytics runtime	Strong computer-vision and ML ecosystem
OpenCV	Video capture and presentation window	Reliable webcam/video decoding and overlays
Ultralytics YOLO 11n	Person detection	Small pretrained detector with a simple inference API
ByteTrack	Temporary multi-object tracking	Associates detections without biometric identity
lap	Assignment solver used by tracking	Required by the ByteTrack adapter on a clean installation
NumPy	Image and numeric arrays	Standard dependency for computer vision
PyYAML	Configuration files	Human-readable camera, tracker, and zone configuration
Next.js / React	Dashboard and API route structure	Component-based interface and server routes
Vinext / Vite	Worker-compatible build	Produces Cloudflare-compatible ESM output
TypeScript	Web contract safety	Catches incorrect event fields during development
Cloudflare Workers	Public web and API runtime	Temporary low-cost hosted endpoint close to users
Cloudflare D1	Event database	Managed SQLite-style durable storage for the pilot
Drizzle	D1 schema and migration tooling	Versioned schema and typed database access
Wrangler	Cloudflare CLI	Login, deploy, bindings, secrets, migrations, and logs
Node test runner	Hosted gateway tests	Lightweight API and rendered HTML verification
Pytest	Vision and analytics tests	Fixtures and focused unit/integration tests
Ruff	Python linting	Fast checks for errors, imports, and style

Important version choices

- Node.js must be at least 22.13.0 according to `package.json`.
- Python must be at least 3.11 according to `pyproject.toml`.
- Ultralytics is pinned to 8.4.95 because TrackBus wraps internal ByteTrack behavior that is protected by adapter tests.
- A pinned version is not automatically the newest or best model. It creates a reproducible baseline.

4. Vision pipeline



Detection versus tracking

Detection answers: "Where is a person in this frame?" Tracking answers: "Is this detection the same temporary person track seen in recent frames?" YOLO performs detection. ByteTrack performs association over time. Track IDs are temporary and exist only inside the running process.

Three-zone event rule

```
OUTSIDE -> DOOR -> INSIDE = BOARDING, occupancy +1  
INSIDE -> DOOR -> OUTSIDE = ALIGHTING, occupancy -1
```

A box appearing inside a zone is not enough. A complete ordered path must be observed for a minimum number of frames. Incomplete, reversed, noisy, direct, or expired transitions are rejected.

Why polygon calibration is necessary

Every bus door, camera angle, image resolution, and lens is different. Default zones are only a safe demo starting point. Real use requires drawing the polygons against the actual doorway view. Bad zones can cause missed crossings even when person detection is correct.

Main causes of the current low recall

- The pretrained general-purpose model is not tuned for the exact bus-door camera angle.
- People are small, partially hidden, blurred, or cropped by the frame edge.
- Several passengers overlap during simultaneous boarding.
- A short track can disappear before it completes all three zones.
- Default confidence, image size, frame skipping, and tracker settings may not fit the footage.
- Zone geometry may not match the actual doorway.

Correct way to measure improvement

Create human-labeled ground truth for complete boardings and alightings, then report precision, recall, F1, false positives, false negatives, and ID fragmentation. Do not report a single informal "accuracy" percentage as a production metric.

5. Canonical JSON event

```
{
  "schemaVersion": "1.0",
  "eventId": "vision-2b7d9f2d",
  "observedAt": "2026-08-21T10:15:30.120Z",
  "source": "vision",
  "busId": "BUS-VIDEO-01",
  "routeId": "DEMO-ROUTE",
  "stopId": "VIDEO-TEST",
  "doorId": "D00R-1",
  "boardings": 1,
  "alightings": 0,
  "occupancy": 1,
  "capacity": 72,
  "confidence": 0.83,
  "qualityFlags": []
}
```

Field meanings

Field	Meaning
schemaVersion	Contract version understood by sender and receiver
eventId	Globally unique retry-safe idempotency key
observedAt	UTC or timezone-aware event time
source	apc, vision, payment, manual, or import
operational IDs	Bus, route, stop, and door context supplied by the operator
boardings	Anonymous boarding delta
alightings	Anonymous alighting delta
occupancy	Reconstructed onboard count after the event
capacity	Configured physical capacity bound
confidence	Source confidence from 0 to 1
qualityFlags	Explainable warnings such as weak detection or synthetic test data

The event intentionally contains no passenger name, face, biometric template, payment card, or persistent person ID.

Where to view JSON

```
https://trackbus-showcase.favi.workers.dev/api/v1/events/passenger-counts?limit=50
```

The operational summary is available at:

```
https://trackbus-showcase.favi.workers.dev/api/v1/operations/pilot
```

6. Windows first-time setup

Run PowerShell inside the repository folder.

6.1 Confirm the correct branch

```
git fetch origin
git checkout agent/trackbus-foundation
git pull --ff-only
git status
```

Why: the strongest integrated TrackBus build is currently on `agent/trackbus-foundation`. `git pull --ff-only` updates without creating an accidental merge commit.

6.2 Confirm runtimes

```
node --version
npm.cmd --version
python --version
```

Expected: Node.js 22.13 or newer and Python 3.11 or newer.

6.3 Install JavaScript dependencies

```
npm.cmd ci
```

Why: `npm ci` installs the exact versions recorded in `package-lock.json`.

6.4 Create an isolated Python environment

```
python -m venv .venv
.\venv\Scripts\python.exe -m pip install --upgrade pip
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

Why: the virtual environment keeps TrackBus packages separate from the Windows system installation. PyTorch and model dependencies can be large, so the first installation may take several minutes.

6.5 Verify the AI packages

```
.\venv\Scripts\python.exe -c "import torch, ultralytics, cv2; print('PyTorch:', torch.__version__); print('CUDA:', torch.cuda.is_available()); print('Ultralytics:', ultralytics.__version__); print('OpenCV:', cv2.__version__)"
```

CUDA: `False` does not prevent the first demonstration. It means inference will run on CPU unless a CUDA-enabled PyTorch build is installed.

7. Run the Vision showcase

7.1 Recommended first run with a video

```
$env:TRACKBUS_API_KEY="YOUR_SAME_CLOUDFLARE_INGEST_SECRET"

.\.venv\Scripts\python.exe showcase.py `
  --video ".\test_video.mp4" `
  --api-url "https://trackbus-showcase.favi.workers.dev/api" `
  --bus-id "BUS-VIDEO-01" `
  --route-id "DEMO-ROUTE" `
  --stop-id "VIDEO-TEST" `
  --door-id "DOOR-1"
```

Why start with video: it is repeatable, avoids webcam permission problems, and makes debugging easier.

7.2 Webcam run

```
$env:TRACKBUS_API_KEY="YOUR_SAME_CLOUDFLARE_INGEST_SECRET"

.\.venv\Scripts\python.exe showcase.py `
  --camera 0 `
  --api-url "https://trackbus-showcase.favi.workers.dev/api" `
  --bus-id "BUS-CAMERA-01" `
  --route-id "DEMO-ROUTE" `
  --stop-id "CAMERA-TEST"
```

Try `--camera 1` if camera 0 is unavailable.

7.3 Local-only run without uploading events

```
.\.venv\Scripts\python.exe showcase.py --video ".\test_video.mp4"
```

7.4 Headless three-frame smoke test

```
.\.venv\Scripts\python.exe showcase.py `
  --video ".\test_video.mp4" `
  --headless `
  --max-frames 3
```

This loads the real model and tracker but does not open the presentation window.

Controls

Key	Action
Space	Pause or resume
F	Toggle fullscreen
O	Toggle information overlay
R	Reset tracks and counters after confirmation
S	Save screenshot
E	Export the event log to CSV
C or Z	Show calibration reminder
Q or Esc	Exit

8. Calibrate the doorway zones

Video calibration

```
.\.venv\Scripts\python.exe showcase.py `
--video ".\test_video.mp4" `
--calibrate `
--save-zones ".\configs\showcase_zones.yaml"
```

Run with saved zones

```
$env:TRACKBUS_API_KEY="YOUR_SAME_CLOUDFLARE_INGEST_SECRET"

.\.venv\Scripts\python.exe showcase.py `
--video ".\test_video.mp4" `
--zones ".\configs\showcase_zones.yaml" `
--api-url "https://trackbus-showcase.favi.workers.dev/api"
```

Calibration rules

- OUTSIDE must cover the approach side of the doorway.
- DOOR must cover the physical crossing corridor.
- INSIDE must cover the bus-interior side.
- Avoid large overlaps between zones.
- Keep zones away from doors, reflections, posters, or areas that cause false person detections.
- Recalibrate if the camera position or resolution changes.

Useful performance overrides

```
.\.venv\Scripts\python.exe showcase.py `
--video ".\test_video.mp4" `
--confidence 0.25 `
--imgsz 960 `
--device auto `
--frame-skip 0
```

Lower confidence can recover missed people but may increase false positives. Larger image size can help small people but reduces FPS. Change one variable at a time and evaluate against labeled events.

9. Cloudflare deployment and operations

What Cloudflare is doing

- Workers runs the website and API routes.
- D1 stores passenger-count event rows.
- Wrangler builds, deploys, binds resources, applies migrations, and manages secrets.
- `TRACKBUS_INGEST_KEY` protects event writes. Public readers never receive that key.

First deployment

```
npx.cmd wrangler login
npx.cmd wrangler secret put TRACKBUS_INGEST_KEY
npm.cmd run cloudflare:dry-run
npm.cmd run cloudflare:deploy
npm.cmd run cloudflare:migrate
```

If Wrangler says the Worker does not exist and asks whether it should create it to add the secret, answer **Y**. Enter your generated ingestion secret only at the hidden prompt. Never put the secret into Git or a screenshot.

If deployment provisions the D1 database, allow Wrangler to write its database ID into the local configuration. Apply the migration only after the binding exists.

Normal update after code changes

```
git pull --ff-only
npm.cmd ci
npm.cmd run cloudflare:dry-run
npm.cmd run cloudflare:deploy
```

The migration command is only required when a new unapplied SQL migration exists.

Production health checks

```
curl.exe https://trackbus-showcase.favi.workers.dev/api/v1/health
curl.exe "https://trackbus-showcase.favi.workers.dev/api/v1/events/passenger-counts?limit=1"
curl.exe https://trackbus-showcase.favi.workers.dev/api/v1/operations/pilot
```

Expected connected state:

- health: **status** is **ok**, **dataMode** is **connected**, and **storageConfigured** is **true**;
- event feed: **live-empty** before the first event, then **live**;
- pilot: **waiting-for-camera** before the first event, then operational bus and source summaries.

10. Send a controlled JSON test event

This proves the API and database independently from the computer-vision quality.

```
$env:TRACKBUS_API_KEY="YOUR_SAME_CLOUDFLARE_INGEST_SECRET"

$event = @{
    schemaVersion = "1.0"
    eventId = "manual-test-$([guid]::NewGuid())"
    observedAt = (Get-Date).ToUniversalTime().ToString("o")
    source = "vision"
    busId = "BUS-JSON-TEST"
    routeId = "DEMO-ROUTE"
    stopId = "API-TEST"
    doorId = "DOOR-1"
    boardings = 1
    alightings = 0
    occupancy = 1
    capacity = 72
    confidence = 0.99
    qualityFlags = @("manual_connectivity_test")
} | ConvertTo-Json

Invoke-RestMethod `
    -Method Post `
    -Uri "https://trackbus-showcase.favi.workers.dev/api/v1/events/passenger-counts" `
    -Headers @{ Authorization = "Bearer $env:TRACKBUS_API_KEY" } `
    -ContentType "application/json" `
    -Body $event
```

Then verify:

```
curl.exe "https://trackbus-showcase.favi.workers.dev/api/v1/events/passenger-counts?limit=5"
```

Expected: the response reports the event as accepted and persisted. The event appears in JSON and in the website's Live Pilot view. Reposting the identical **eventId** should be treated as a duplicate rather than increasing occupancy twice.

11. Exact PowerShell verification commands

Web lint, build, API, and rendered HTML

```
npm.cmd run lint
npm.cmd test
npm.cmd run cloudflare:dry-run
```

What these prove:

- TypeScript and JavaScript source passes lint rules.
- The Vinext production Worker artifact builds.
- The health and forecast endpoints respond correctly.
- Invalid and unauthenticated events are rejected.
- An authenticated event is stored once and can be read back.
- The pilot endpoint reconstructs its operational view.
- Final HTML contains TrackBus metadata, navigation, and favicon URL.

Vision lint and tests

```
.\.venv\Scripts\python.exe -m ruff check showcase.py trackbus tests
.\.venv\Scripts\python.exe -m pytest -q
```

These cover configuration, calibration, zones, tracking adapters, trajectory counting, event contracts, API retries, diagnostics, evaluation, and showcase layout helpers.

Analytics service checks

```
Push-Location ".\services\analytics-api"
..\..\venv\Scripts\python.exe -m pip install -e ".[dev]"
..\..\venv\Scripts\python.exe -m ruff check src tests
..\..\venv\Scripts\python.exe -m pytest -q
Pop-Location
```

Replay-tool tests

```
$env:PYTHONPATH="tools"
.\.venv\Scripts\python.exe -m unittest discover -s tools/tests -v
Remove-Item Env:PYTHONPATH
```

Current verification recorded for this guide

Check	Result on 21 Aug 2026
Web lint	PASS, zero lint errors
Production build	PASS, six routes emitted and Worker artifact validated
Web/API/HTML tests	PASS, 7 passed and 0 failed
Replay tool	PASS, 2 passed and 0 failed
Vision tests	PASS, 250 passed and 0 failed; 4 expected configuration warnings
Analytics API tests	PASS, 18 passed and 0 failed; 1 dependency deprecation warning
Python lint	PASS for Vision and analytics service

For a presentation, run the same commands again on the exact laptop and commit being demonstrated and show the terminal output.

12. Five-minute technical demonstration

Before the meeting

1. Pull the branch and confirm `git status` is clean.
2. Run the automated tests and save the output.
3. Open the website and the raw JSON endpoint.
4. Set `TRACKBUS_API_KEY` in the same PowerShell session.
5. Use a short repeatable video before trying the webcam.
6. Calibrate the zones for that video.

Suggested speaking script

0:00 - Problem: "Transport data arrives from separate sensors and systems. TrackBus normalizes anonymous passenger events into one operational picture."

0:40 - Vision: Start the video and say: "YOLO detects people, ByteTrack creates temporary process-local tracks, and the event layer requires a complete three-zone crossing. There is no facial recognition."

1:40 - JSON: Open the raw endpoint and say: "The edge sends count events, not video. Each event carries operational IDs, count deltas, occupancy, confidence, and quality flags."

2:30 - Database and dashboard: Open Live Pilot and say: "The Worker validates the bearer secret and contract, D1 stores the event idempotently, and the dashboard refreshes the operational view."

3:30 - Reliability and privacy: Explain the offline queue, stable event ID, no biometric identity, and edge-local raw video.

4:15 - Honest limitation: "The current general model missed many passengers in our sample. The recent informal observation suggests only about 20-30% recall. The architecture works, but production accuracy requires labeled data, calibration, model tuning, and a measured pilot."

4:50 - Close: "The next milestone is not a bigger dashboard. It is a controlled 30-60 day pilot that measures counting quality and data reliability before operational claims."

13. Technical Q&A - core answers

Q1. What exactly is TrackBus?

It is a transport data normalization and operational intelligence platform. Passenger counting is one source capability. The platform combines events into occupancy, health, reconciliation, forecasts, and reviewable operator actions.

Q2. Is it only an AI camera project?

No. Vision is an optional edge adapter. The canonical contract also supports APC sensors, payments, imports, and manual corrections. This reduces dependency on one hardware vendor.

Q3. Why YOLO?

YOLO provides a practical pretrained person detector with good tooling and real-time variants. The nano model is a small starting point for edge hardware. It is not considered production-accurate without evaluation on the target bus footage.

Q4. Why ByteTrack?

Counting requires continuity across frames. ByteTrack associates detections into temporary tracks, including some lower-confidence detections, without using faces or persistent identities.

Q5. How is direction determined?

The configured anchor point of a temporary track must visit the zones in a stable order. OUTSIDE -> DOOR -> INSIDE means boarding. The reverse means alighting.

Q6. How do you prevent double counting?

The local state machine emits one event per confirmed track transition. The API also uses `eventId` as an idempotency key, so retries of the same event do not create a second stored event.

Q7. What happens if the internet fails?

The edge client saves the canonical event to a disk queue and retries it later with the same event ID. Detection and counting can continue locally.

Q8. Does the website receive camera video?

No. The normal contract sends numeric count events and metadata. Raw frames stay at the edge by default.

14. Technical Q&A - specialist questions

Q9. What is the current accuracy?

There is no valid production accuracy number yet. In the latest informal video run, visible recall looked roughly 20-30% because many people were missed. That is not a labeled evaluation. The correct next step is ground-truth annotation and precision, recall, and F1 reporting by camera condition.

Q10. Why did it miss people?

Likely causes include occlusion, small targets, motion blur, frame edges, general-model domain mismatch, track fragmentation, and zones that do not match the doorway. These hypotheses must be separated by diagnostics rather than guessed from one total count.

Q11. Would lowering the confidence solve it?

It may improve recall but can increase false positives and noisy tracks. Confidence, image size, frame rate, detector size, tracker thresholds, and zone geometry must be evaluated together against labeled crossings.

Q12. Why not identify passengers with faces?

Identity is unnecessary for counting and creates major privacy and governance risk. Temporary track IDs provide enough short-term continuity for a doorway transition and are discarded when the process ends.

Q13. Why Cloudflare D1?

D1 is convenient for a temporary pilot because it is managed, works directly with Workers, and supports SQL-style durable storage. It is not an irreversible choice because the canonical event contract separates storage from sources.

Q14. Is the API secure?

Event writes require a bearer secret stored as a Cloudflare Worker secret. The API validates the contract and rejects unauthorized writes. For production, add secret rotation, per-device credentials, rate limits, audit alerts, stricter public-read policy, and environment separation.

Q15. Can this scale to many buses?

The event contract and stateless ingestion boundary can scale horizontally, but production scale must test Worker limits, database query patterns, event volume, retries, device provisioning, monitoring, and retention. The current deployment is a pilot, not a proven city-scale capacity claim.

Q16. Are dashboard forecasts real?

No. The Live Pilot event view is connected to stored events. Forecast, dispatch, and passenger scenarios are explicitly synthetic demonstrations until representative data and evaluation exist.

15. Technical Q&A - ownership and roadmap

Q17. What did you build versus what comes from libraries?

YOLO, ByteTrack, React, Workers, and D1 are established technologies. TrackBus integrates them and adds the camera configuration, exclusion and zone handling, stable path state machine, occupancy rules, canonical transport event, retry client, API validation, D1 schema, operational read models, dashboard, and verification suites.

Q18. What is the most important engineering decision?

The strongest decision is the canonical vendor-neutral event boundary. It prevents the product from becoming a single-camera demo and allows other approved passenger-count sources to use the same analytics and interface.

Q19. What would you do before a real pilot?

Select one repeatable door and camera, obtain permission, calibrate the scene, label representative footage, define measurable acceptance criteria, measure day/night and crowd conditions, add device-specific credentials and monitoring, and document retention and incident procedures.

Q20. What are reasonable pilot metrics?

- Boarding and alighting precision, recall, and F1.
- Occupancy error over time and at terminal resets.
- Event delivery success and retry delay.
- Source freshness and camera uptime.
- Percentage of events carrying quality flags.
- Reconciliation gap against a trusted manual sample or approved APC.
- Operator time needed to investigate flagged periods.

Q21. What should not be claimed today?

Do not claim production-grade accuracy, city-wide readiness, guaranteed savings, safety certification, biometric identification, or a validated forecasting advantage. Demonstrate the working pipeline and clearly separate real evidence from the product vision.

16. Troubleshooting

Symptom	Likely cause	Action
python not found	Python not installed or PATH missing	Install Python 3.11+ and reopen PowerShell
node, npm, or npx not found	Node.js not installed or terminal not refreshed	Install Node LTS and reopen PowerShell
PyTorch is not installed	Python requirements not installed in the interpreter being used	Run the <code>.venv</code> installation commands and launch with <code>.venv\Scripts\python.exe</code>
Could not open video source	Wrong path, codec issue, or camera busy	Quote the full path, try camera 1, close other camera apps
Model downloads on first run	YOLO weights absent locally	Allow the first download; later runs use the cached file
People detected but no counts	Crossing did not complete configured zones	Calibrate zones and inspect track trails
Low recall	Domain mismatch, occlusion, resolution, thresholds	Label footage and evaluate controlled changes
Events remain in offline-queue	API URL, connectivity, or secret mismatch	Check health URL and reset the PowerShell secret
HTTP 401	Wrong or missing ingestion secret	Set <code>TRACKBUS_API_KEY</code> to the same Worker secret
HTTP 422	JSON contract invalid	Compare all fields with the canonical example
Site says live-empty	D1 works but no event has been stored	Run the controlled JSON test or a confirmed crossing
D1 database_id missing	Remote database binding not provisioned	Deploy once, allow provisioning, then apply migration
Wrangler says Worker does not exist	Secret was added before first Worker creation	Answer Y when Wrangler offers to create it
Favicon still old	Browser cache	Hard refresh, open a private window, or wait for CDN/browser cache expiry

17. Security, privacy, and operational checklist

Before every demonstration

- Use footage you are allowed to process.
- Do not expose the ingestion secret in screenshots or command history.
- Use a dedicated test bus ID and route ID.
- Explain that temporary track IDs are not identities.
- Keep raw video local unless an approved evidence process says otherwise.
- Clearly label manual JSON test events with a quality flag.
- Do not mix synthetic forecast data with real pilot evidence.

Before a production pilot

- Create separate development, pilot, and production environments.
- Issue a unique credential per edge device and support rotation/revocation.
- Restrict public event reads or aggregate them where appropriate.
- Add retention rules for events, logs, screenshots, and any approved footage.
- Add rate limits, alarms, uptime monitoring, and audit logging.
- Define who can reset occupancy and how resets are recorded.
- Complete a privacy and security review with the transport operator.
- Define measurable go/no-go thresholds before collecting results.

18. Glossary and quick memory sheet

Term	Short definition
APC	Automatic Passenger Counting sensor or system
Detection	Finding a person bounding box in one frame
Tracking	Associating detections across frames
Track ID	Temporary process-local identifier, not passenger identity
State machine	Rules that accept only a valid ordered zone path
Recall	Share of real crossings that the system finds
Precision	Share of predicted crossings that are correct
F1	Balance between precision and recall
Idempotency	Repeating the same event does not duplicate its effect
Canonical contract	Shared JSON format used by every approved source
Edge	Local bus/camera computer where video is processed
Worker	Cloudflare serverless program hosting the site and API
D1	Cloudflare's managed SQLite-style database
Migration	Versioned SQL change applied to the database schema
Wrangler	Cloudflare command-line deployment and operations tool
Synthetic	Generated demonstration data, not measured production evidence

Seven sentences to remember

1. TrackBus is a vendor-neutral transport intelligence platform, not only a camera counter.
2. YOLO detects people and ByteTrack maintains temporary tracks.
3. A complete three-zone path determines boarding or alighting.
4. The edge sends anonymous JSON events, not raw video.
5. A Worker validates events, D1 stores them idempotently, and Live Pilot displays them.
6. Forecast scenarios are synthetic, while stored camera events are real.
7. Current counting quality is a prototype limitation; production claims require labeled evaluation and a controlled pilot.

Primary links

- Dashboard: <https://trackbus-showcase.favi.workers.dev>
- Health: <https://trackbus-showcase.favi.workers.dev/api/v1/health>
- JSON events:
<https://trackbus-showcase.favi.workers.dev/api/v1/events/passenger-counts?limit=50>
- Pilot summary: <https://trackbus-showcase.favi.workers.dev/api/v1/operations/pilot>
- Source branch: [agent/trackbus-foundation](#)

End of guide. Keep this document with the exact commit used for each demonstration.